```
title: HashMap
tags: 集合 源码 面试
created: 2022-07-25 16:22:07
modified: 2022-07-29 01:33:30
```

整体结构

HashMap 底层的数据结构主要是：`数组 + 链表 + 红黑树` 。其中，当链表的长度大于等于8 并且数组的长度大于64 时，链表会转化为红黑树，而当红黑树的节点个数小于等于6 时，红黑树为转化为链表。

![[../../Excalidraw/HashMap 数据结构.excalidraw]]

重要属性

```java
// 初始容量
static final int DEFAULT_INITIAL_CAPACITY = 1 << 4;

// 最大容量
static final int MAXIMUM_CAPACITY = 1 << 30;

// 负载因子默认值
static final float DEFAULT_LOAD_FACTOR = 0.75f;

// bin(桶)容量大于等于8时，链表转化成红黑树
static final int TREEIFY_THRESHOLD = 8;

// bin(桶)容量小于等于6时，红黑树转化成链表
static final int UNTREEIFY_THRESHOLD = 6;

// 数组容量最小64时才会转会成红黑树
static final int MIN_TREEIFY_CAPACITY = 64;

// 用于fail-fast的，记录HashMap结构发生变化(数量变化或rehash)的数目
transient int modCount;

// HashMap 的实际大小，可能不准(因为当你拿到这个值的时候，可能又发生了变化)
transient int size;

// 扩容的门槛，如果初始化时，给定数组大小的话，会通过 tableSizeFor 方法计算，得到一个接近于 2 的幂次方的数组大小
// 如果是通过 resize 方法进行扩容后，大小 = 数组容量 * 0.75
int threshold;

// 存放数据的数组
transient Node<K,V>[] table;

// bin node 节点
static class Node<K,V> implements Map.Entry<K,V> {
    final int hash; // key的hash值
    final K key; // key值
    V value; // value值
    Node<K,V> next; // 当前节点的下一个节点

//红黑树的节点
static final class TreeNode<K,V> extends LinkedHashMap.Entry<K,V> {
    TreeNode<K,V> parent; // 当前节点的父节点
    TreeNode<K,V> left; // 当前节点的左节点
    TreeNode<K,V> right; // 当前节点的右节点
    TreeNode<K,V> prev; // 当前节点在双向链表中的上一个节点
    boolean red; // 当前节点的颜色(红/黑)
```

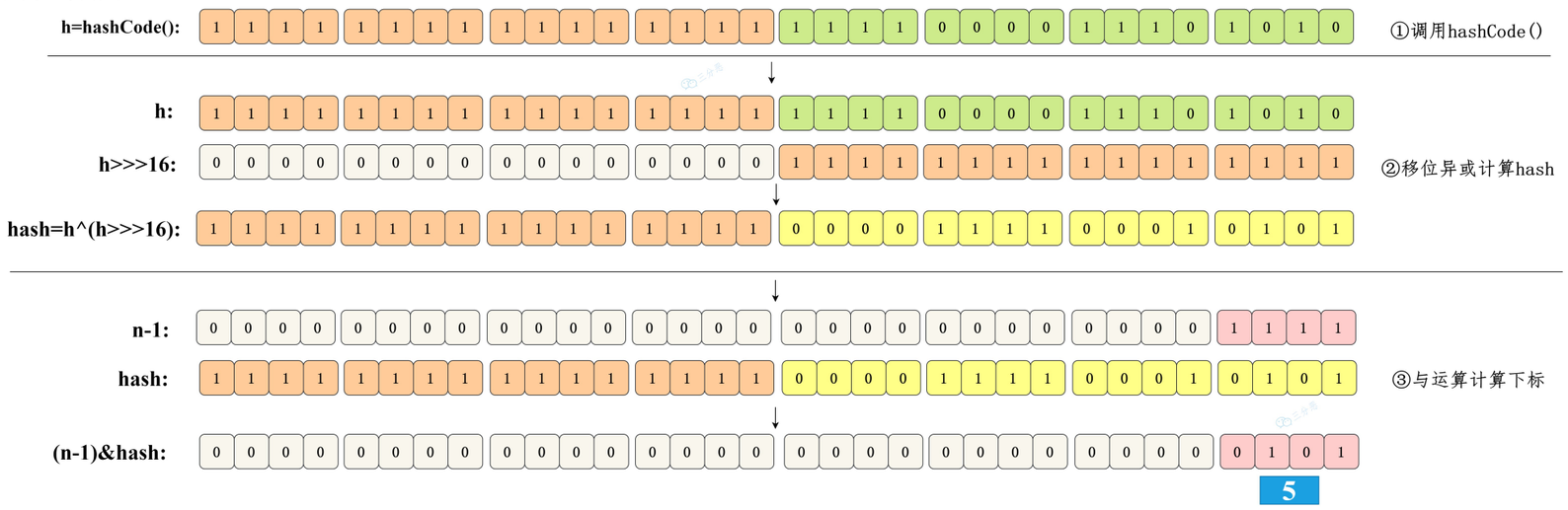
hash 函数

```java
static final int hash(Object key) {
    int h;
    return (key == null) ? 0 : (h = key.hashCode()) ^ (h >>> 16);
}
```

在 hash 方法中，hash 值等于将 key 的 hashCode 值右移 16 位后与 hashCode 本身进行按位异或操作。也就是让低16 位与高16 位进行异或，高16 位保持不变(与0 异或都是自己本身)，让高位也得以参与散列运算，使得散列更加均匀。

计算数组下标 index

数组下标 index 如何确定？使用 (n-1) & hash，其中 n 为数组长度，如默认的初始化容量为 16，即数组长度 n = 16，那么 n - 1 = 15，换算成二进制为 00001111，那么 (n-1)&hash 取的是 hash 值的低四位，4 个 bit 位的最大值为 15，所以往数组中存放元素时的数组下标就可以使用该方式确定。



resize 扩容方法

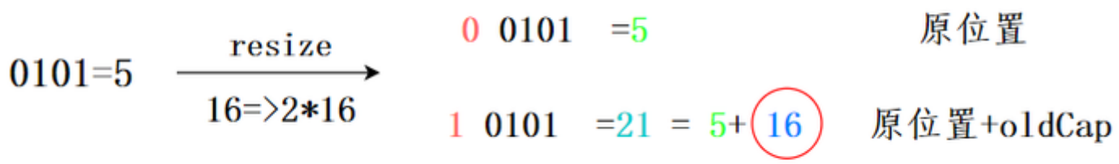
在HashMap 源码中，把数组的初始化操作也放到了扩容方法中，因而扩容方法源码主要分为两部分：确定新的数组大小、迁移数据。

```java
final Node<K,V>[] resize() {
    // 将原来的table交由oldTab保存
    Node<K,V>[] oldTab = table;
    // 获取原来的数组长度赋值给oldCap
    int oldCap = (oldTab == null) ? 0 : oldTab.length;
    // 将原来的阈值大小threshold交由oldThr保存
    int oldThr = threshold;
    // 创建变量 newCap->新的数组大小，newThr->新的阈值大小
    int newCap, newThr = 0;
    if (oldCap > 0) {// 如果原来的数组长度大于0
        if (oldCap >= MAXIMUM_CAPACITY) {// 如果原来的数组长度已经大于等于最大的数组长度(1<<30)
            // 直接把阈值大小设置为最大整数2^31-1
            threshold = Integer.MAX_VALUE;
            // 返回原来的数组
            return oldTab;
        }
        else if ((newCap = oldCap << 1) < MAXIMUM_CAPACITY &&
                 oldCap >= DEFAULT_INITIAL_CAPACITY) // oldCap<<1 => 将原来的数组大小*2
            newThr = oldThr << 1; // 将阈值也扩大1倍 <=> oldThr * 2
    }
    else if (oldThr > 0) // 当原来的阈值大于0但数组长度=0时，对应的情况就是使用带有指定数组长度和加载因子的构造器创建HashMap
        // 新的数组长度等于原来的阈值大小
        newCap = oldThr;
    else { // 对应的情况是使用默认的构造器创建HashMap
        // 默认的数组大小为16
        newCap = DEFAULT_INITIAL_CAPACITY;
        // 默认的阈值大小 = 16 * 0.75(加载因子) = 12
        newThr = (int)(DEFAULT_LOAD_FACTOR * DEFAULT_INITIAL_CAPACITY);
    }
    if (newThr == 0) { // 当新的阈值大小为0时，对应的情况就是使用带有指定数组长度和加载因子的构造器创建HashMap
        float ft = (float)newCap * loadFactor;
        newThr = (newCap < MAXIMUM_CAPACITY && ft < (float)MAXIMUM_CAPACITY ?
                  (int)ft : Integer.MAX_VALUE);
    }
    // 将新的阈值大小赋值给threshold
    threshold = newThr;
    @SuppressWarnings({"rawtypes","unchecked"})
    // 此时扩容已经完成，接下来的工作就是进行数据拷贝，将原数组中的数据迁移到新数组中
    // 创建新的数组，数组长度为上面计算出来的新的数组大小
    Node<K,V>[] newTab = (Node<K,V>[])new Node[newCap];
    // table指向新数组
    table = newTab;
    if (oldTab != null) {
        for (int j = 0; j < oldCap; ++j) { // 遍历原来的数组
            Node<K,V> e; // 当前节点变量
            if ((e = oldTab[j]) != null) { // 判断数组中当前节点是否为null，即判断原来数组中该位置是否有值
                // 将原来数组中该位置的节点清空
                oldTab[j] = null;
                if (e.next == null) // 判断当前节点是否有下一个节点，如果为null，则表示当前位置只存在一个节点，所以只需要计算该节点
位于新数组中的数组下标即可
                    // 那么再次使用 hash & (n - 1) 来计算当前节点位于新数组中的数组下标，在新数组的该位置存放当前节点
                    newTab[e.hash & (newCap - 1)] = e;
                // 该节点存在下一个节点，所以有可能是链表或者红黑树结构
                else if (e instanceof TreeNode) // 判断当前节点是不是树节点，即判断当前位置是否已经转化为红黑树结构
```

```
                    ((TreeNode<K,V>)e).split(this, newTab, j, oldCap);
                else { // 表明该节点是一个链表结构
                    // 新的位置只有两种可能：原位置，原位置+老数组长度
                    // 把原链表拆成两个链表，然后再分别插入到新数组的两个位置上
                    // loXXX表示数组下标位置不变的链表，低位链表
                    Node<K,V> loHead = null, loTail = null;
                    // hiXXX表示原来数组下标+原来数组长度的位置处的链表，高位链表
                    Node<K,V> hiHead = null, hiTail = null;
                    // next节点，用于递归该位置的链表
                    Node<K,V> next;
                    do {
                        // next节点指向当前节点的下一个节点
                        next = e.next;
                        if ((e.hash & oldCap) == 0) { // 使用hash & 原来的数组长度大小(如16 = 10000)，如果最高位为0，表示该节点位于原位置，即存在于低位链表中
                            if (loTail == null) // 判断低位链表的尾节点是否为null，如果为null，表示当前低位链表还没有节点，则将低位链表的头节点指向当前节点
                                loHead = e; // 将低位链表的头节点指向当前节点，作为该链表的第一个节点
                            else // 如果不为null，则低位链表的尾节点的下一个节点指向当前节点，即将整个低位链表串起来
                                loTail.next = e;
                            loTail = e; // 使尾节点指向当前节点，即移动低位链表的尾节点
                        }
                        else {
                            if (hiTail == null)
                                hiHead = e;
                            else
                                hiTail.next = e;
                            hiTail = e;
                        }
                    } while ((e = next) != null); // 递归当前链表直至结束
                    if (loTail != null) { // 如果低位链表不为空
                        // 将低位链表的尾节点的下一个节点清空
                        loTail.next = null;
                        // 在新数组的原位置处放入低位链表
                        newTab[j] = loHead;
                    }
                    if (hiTail != null) { // 如果高位链表不为空
                        // 将高位链表的尾节点的下一个节点清空
                        hiTail.next = null;
                        // 在新数组的原位置+原来数组长度大小处放入高位链表
                        newTab[j + oldCap] = hiHead;
                    }
                }
            }
        }
    }
    // 返回新的数组
    return newTab;
}
```

详细说明一下节点在扩容时位于原位置还是原位置+ 原来数组长度的位置，这个结论是怎么来的？以及HashMap 源码是如何来判断的？ 首先看下下面这张图，图 `a` 表示扩容前 key1 和 key2 确定数组下标的位置，图 `b` 表示扩容后 key1 和 key2 确定数组下标的位置。

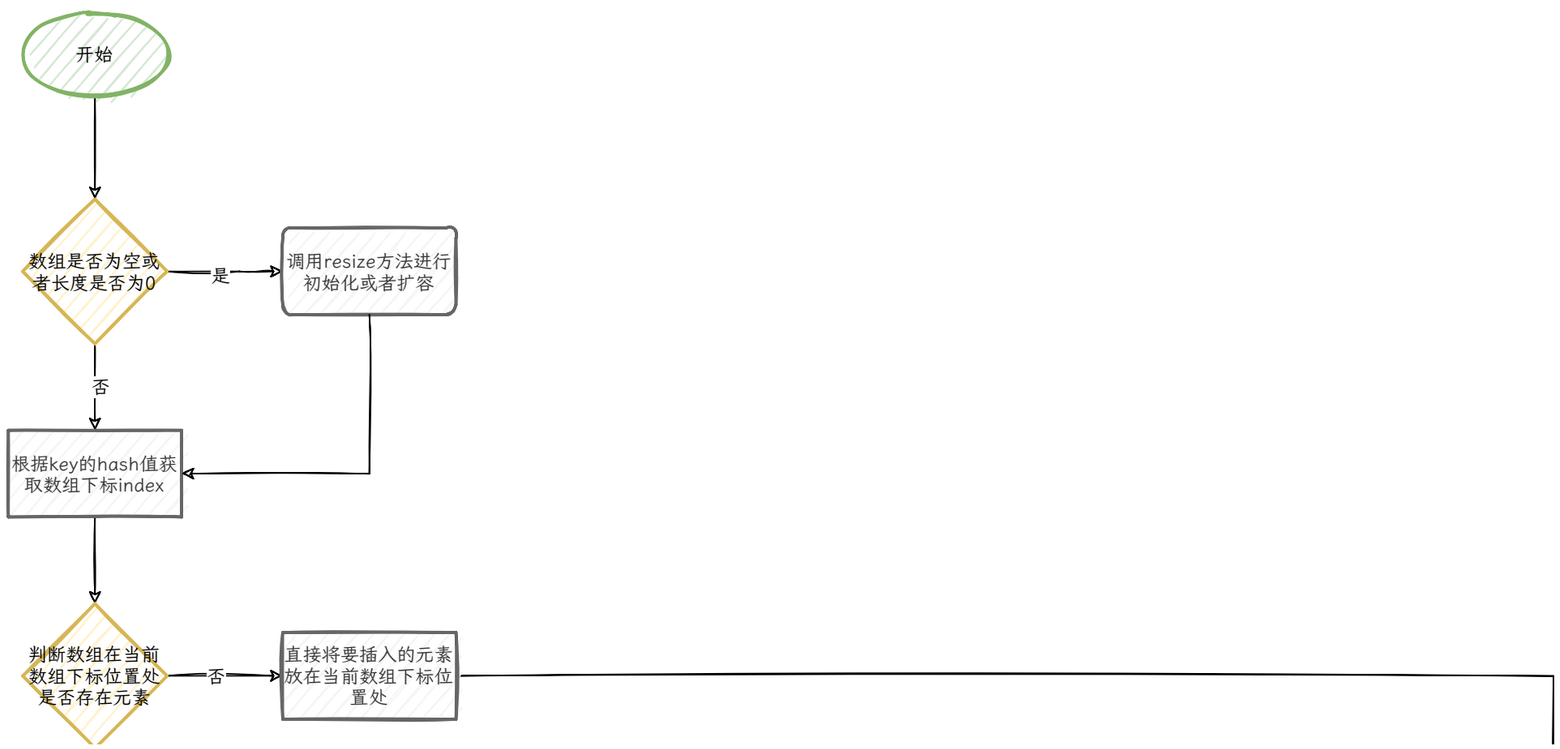
数组在扩容后，因为数组长度 n 变为原来的 2 倍，即 n - 1 就会比原来在高位处多 1 bit 位，因此新的数组下标就会发生这样的变化：

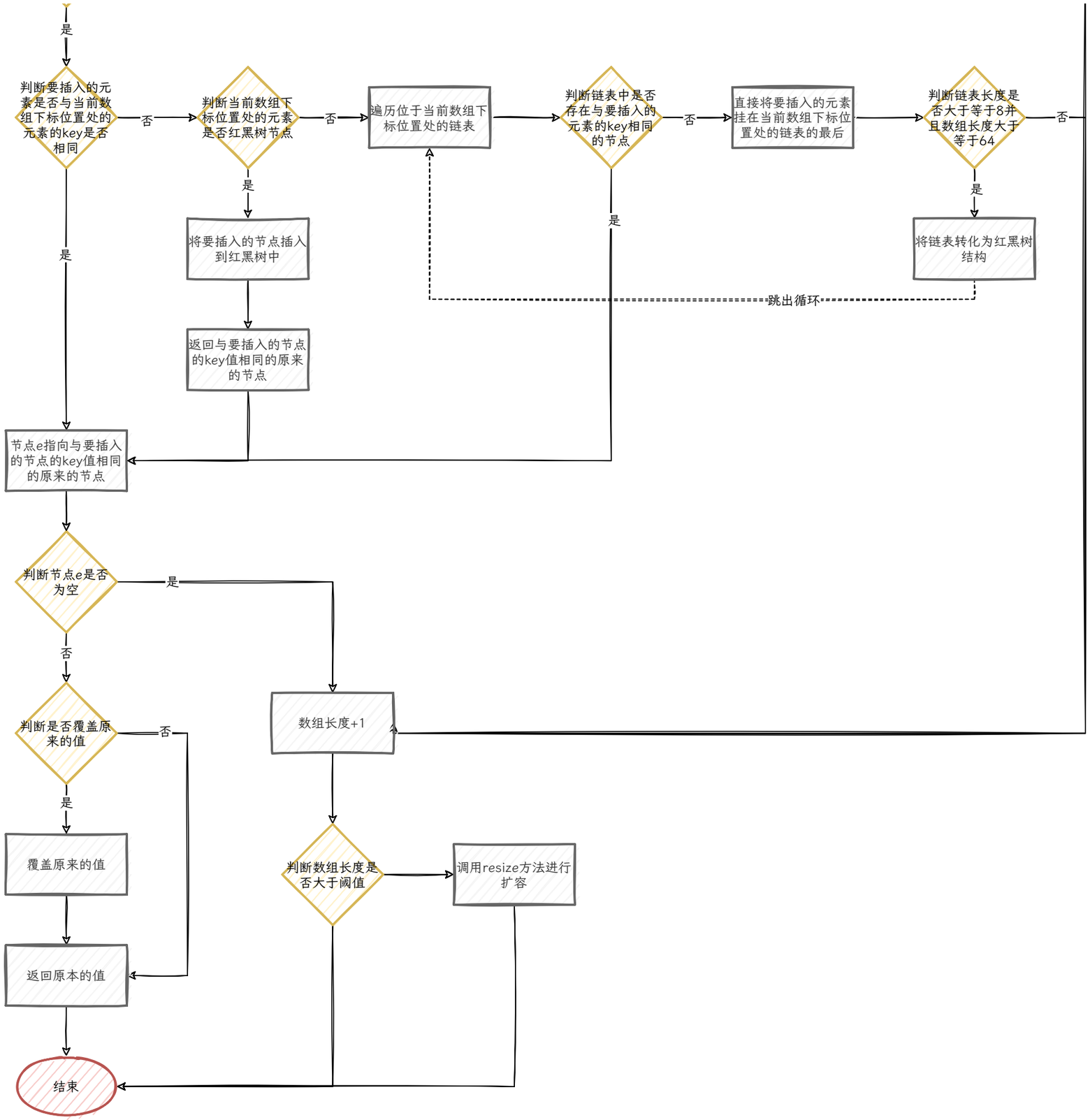


所以在扩容时，只需要看 hash 值在原数组长度所对应二进制的最高位的位置是 0 还是 1，0 的话表示还是原位置，1 的话表示当前节点所在新数组的位置=原位置 + 原来的数组长度。

put 添加节点

流程




是
判断要插入的元素是否与当前数组下标位置处的元素的key是否相同
否
判断当前数组下标位置处的元素是否红黑树节点
否
遍历位于当前数组下标位置处的链表
判断链表中是否存在与要插入的元素的key相同的节点
否
直接将要插入的元素挂在当前数组下标位置处的链表的最后
判断链表长度是否大于等于8并且数组长度大于等于64
否
是
将要插入的节点插入到红黑树中
返回与要插入的节点的key值相同的原来的节点
是
将链表转化为红黑树结构
跳出循环
是
节点e指向与要插入的节点的key值相同的原来的节点
判断节点e是否为空
是
否
判断是否覆盖原来的值
否
是
覆盖原来的值
返回原本的值
结束
数组长度+1
判断数组长度是否大于阈值
调用resize方法进行扩容

代码分析

```java
// 参数onlyIfAbsent表示是否覆盖原来的值，true表示不覆盖，false表示覆盖，默认为false
final V putVal(int hash, K key, V value, boolean onlyIfAbsent, boolean evict) {
    // tab是存放节点的数组，n是数组长度,i是当前要插入的节点在数组中的下标，p是当前数组下标处的节点
    Node<K,V>[] tab; Node<K,V> p; int n, i;
    if ((tab = table) == null || (n = tab.length) == 0) // tab指向全局的table数组，判断tab是否为null或者tab的长度为0
        // 调用resize扩容方法初始化tab或者给tab扩容
        n = (tab = resize()).length;
    if ((p = tab[i = (n - 1) & hash]) == null) // hash & (n-1) => 前要插入节点在数组中的数组下标index，p指向数组中当前位置的节点，
判断当前位置是否不存在节点
        // 当前位置没有节点，则直接在当前位置插入节点
        tab[i] = newNode(hash, key, value, null);
    else { // 当前位置存在节点
        // e指向要插入节点在当前位置处的节点
        Node<K,V> e; K k;
        if (p.hash == hash &&
            ((k = p.key) == key || (key != null && key.equals(k)))) // 判断当前要插入节点的hash值和key值是否与当前位置处已有节点
的hash值和key值相同
            // 如果相同，表示要插入的节点与当前位置处已有节点是同一个节点，
            e = p;
        else if (p instanceof TreeNode) // 判断该节点是否树节点
            // 如果是的话，则使用红黑树的方法插入节点
            e = ((TreeNode<K,V>)p).putTreeVal(this, tab, hash, key, value);
        else { // 最后一种情况，在当前位置处的链表中插入
            for (int binCount = 0; ; ++binCount) { // 遍历当前位置处的链表
                // e指向p的下一个节点，在循环体的最后又将p指向e，如此这般可以用来遍历当前链表
                if ((e = p.next) == null) { // 判断p节点的下一个节点是否为null
                    // 如果p节点没有下一个节点，则直接将要插入的节点挂在p节点后面
                    p.next = newNode(hash, key, value, null);
                    if (binCount >= TREEIFY_THRESHOLD - 1) // 如果当前链表长度已经大于等于8个
                        // 在treeifyBin方法中还会继续判断数组长度是否小于64
                        // 如果小于的话，则优先进行扩容而不是树化；如果数组长度大于等于64，则将链表转化为红黑树结构
                        treeifyBin(tab, hash);
                    // 跳出循环，停止链表遍历
                    break;
                }
                if (e.hash == hash &&
                    ((k = e.key) == key || (key != null && key.equals(k)))) // 判断当前要插入节点的hash值和key值是否与正在e节点的
hash值和key值相同
                    // 如果相同，表示要插入的节点与e节点的key相同，则不执行插入操作，跳出循环，停止链表遍历
                    break;
                p = e;
            }
        }
        if (e != null) { // 判断e是否为null，如果不为null，则表示在当前链表中找到与要插入节点的key相同的节点
            // 取出e节点的value值赋值给oldValue
            V oldValue = e.value;
            if (!onlyIfAbsent || oldValue == null)  // 判断onlyIfAbsent是否为false或者原来的值是否为null
                // 如果onlyIfAbsent为false或者原来的值为null，则用要插入节点的value值覆盖掉原来的值
                e.value = value;
            afterNodeAccess(e);
            // 返回原来的值
            return oldValue;
        }
    }
```

```
    // modCount+1表示HashMap此时结构已经发生变化
    ++modCount;
    if (++size > threshold) // 判断节点数量是否已经大于阈值
        // 如果大于的话，则调用resize方法进行扩容操作
        resize();
    afterNodeInsertion(evict);
    // 当前要插入的节点是一个新的节点，在原来的数组+链表+红黑树中不存在，当然原来的值也就不存在，返回null
    return null;
}
```

红黑树

红黑树

链表树化

```
// tab是当前存放集合数据的数组，hash是要插入节点的key所对应的hash值
final void treeifyBin(Node<K,V>[] tab, int hash) {
    // n是数组长度，index是要插入节点在数组中的数组下标，e指向遍历链表时的节点
    int n, index; Node<K,V> e;
    if (tab == null || (n = tab.length) < MIN_TREEIFY_CAPACITY) // 判断tab是否为空或者tab的长度小于最小树化容量64
        // 如果tab为空或者数组的长度小于64，则先不进行树化而是扩容
        resize();
    else if ((e = tab[index = (n - 1) & hash]) != null) { // hash & (n-1) => 当前要插入节点在数组中的数组下标index，e指向数组中当
前位置的节点，判断当前位置是否存在节点
        // 满足当前数组下标位置处存在节点
        // hd是头节点，tl是尾节点
        TreeNode<K,V> hd = null, tl = null;
        do { // 循环遍历链表，其实循环体中的代码就是将链表中的节点转化为红黑树节点并且形成一个双向链表的结构
            TreeNode<K,V> p = replacementTreeNode(e, null);
                hd = p; // 头节点指向链表中的第一个节点
            else {
                p.prev = tl;
                tl.next = p;
            }
            // 尾节点指向当前正在遍历的节点
            tl = p;
        } while ((e = e.next) != null);
        if ((tab[index] = hd) != null) // 将上面形成的双向链表结构放在当前数组下标位置处的数组中，并且判断头节点是否不为null
            // 执行TreeNode中treeify方法将链表转化为真正的红黑树结构
            hd.treeify(tab);
    }
}
```

```
[HashMap二十三问 · 语雀](https://www.yuque.com/lovebetterworld/ioayz6/qx4frz#SpUEn)
```